Usage

Basic setup
lspconfig: https://github.com/neovim/nvim-lspconfig
List of servers: https://github.com/neovim/nvim-lspconfig/blob/master/doc/server_configurations.md

Recommended keybindings and completion
Keybinding (aka shortcut)
Completion
By default, neovim does not provide any settings for using LSP.
In particular,
- **keybindings**: i.e. mappings to LSP functions
e.g. jump to definition
- **completion**: completion using LSP
e.g. omnicomplete
Need to be configured manually via the **on_attach** handler.
Recommended: https://github.com/neovim/nvim-lspconfig#keybindings-and-completion

```
-- Initialization:
-- Set LSP settings for usage in current buffer.
-- e.g. set omnifunc for completion
-- e.g. define key mappings to lsp functions
local on_attach = function(client, bufnr)
  local function buf_set_keymap(...) vim.api.nvim_buf_set_keymap(bufnr, ...)
end
  local function buf_set_option(...) vim.api.nvim_buf_set_option(bufnr, ...)
end

  -- note: indicator
  -- exists('yt_lspconfig') to check if initialized
  vim.api.nvim_buf_set_var(bufnr, 'yt_lspconfig', 1)

  -- note: completion
  -- i.e. manually triggered by <c-x><c-o>
  buf_set_option('omnifunc', 'v:lua.vim.lsp.omnifunc')

  -- note: mappings
  local opts = { noremap=true, silent=true }
  buf_set_keymap('n', 'gD', '<cmd>lua vim.lsp.buf.declaration()<CR>',
opts)
  buf_set_keymap('n', 'gd', '<cmd>lua vim.lsp.buf.definition()<CR>', opts)
  buf_set_keymap('n', 'K', '<cmd>lua vim.lsp.buf.hover()<CR>', opts)
  buf_set_keymap('n', 'gi', '<cmd>lua vim.lsp.buf.implementation()<CR>',
opts)
  buf_set_keymap('n', '<C-k>', '<cmd>lua vim.lsp.buf.signature_help()<CR>',
opts)
  buf_set_keymap('n', '<leader>wa', '<cmd>lua
vim.lsp.buf.add_workspace_folder()<CR>', opts)
  buf_set_keymap('n', '<leader>wr', '<cmd>lua
vim.lsp.buf.remove_workspace_folder()<CR>', opts)
  buf_set_keymap('n', '<leader>wl', '<cmd>lua
print(vim.inspect(vim.lsp.buf.list_workspace_folders()))<CR>', opts)
  buf_set_keymap('n', '<leader>D', '<cmd>lua
vim.lsp.buf.type_definition()<CR>', opts)
  buf_set_keymap('n', '<leader>rn', '<cmd>lua vim.lsp.buf.rename()<CR>',
opts)
  buf_set_keymap('n', '<leader>ca', '<cmd>lua vim.lsp.buf.code_action()<CR>',
opts)
  buf_set_keymap('n', 'gr', '<cmd>lua vim.lsp.buf.references()<CR>', opts)
```

```
  buf_set_keymap('n', '<leader>Ld', '<cmd>lua
vim.lsp.diagnostic.show_line_diagnostics()<CR>', opts)
  buf_set_keymap('n', '[d', '<cmd>lua vim.lsp.diagnostic.goto_prev()<CR>',
opts)
  buf_set_keymap('n', ']d', '<cmd>lua vim.lsp.diagnostic.goto_next()<CR>',
opts)
  buf_set_keymap('n', '<leader>Ll', '<cmd>lua
vim.lsp.diagnostic.set_loclist()<CR>', opts)
  buf_set_keymap('n', '<leader>Lf', '<cmd>lua vim.lsp.buf.formatting()<CR>',
opts)

end

-- The following example advertise capabilities to `clangd`.
-- require'lspconfig'.flow.setup { capabilities = capabilities }
-- require'lspconfig'.tsserver.setup { capabilities = capabilities }
-- Use a loop to conveniently call 'setup' on multiple servers and
-- map buffer local keybindings when the language server attaches
local servers = {
 flow = { … custom settings … },
 bashls = { … custom settings … },
}
for name, settings in ipairs(servers) do
  vim.lsp.enable(name)
  vim.lsp.config(name, {
    capabilities = capabilities,
    on_attach = on_attach,
    flags = {
      debounce_text_changes = 150,
    }
  })
end
```

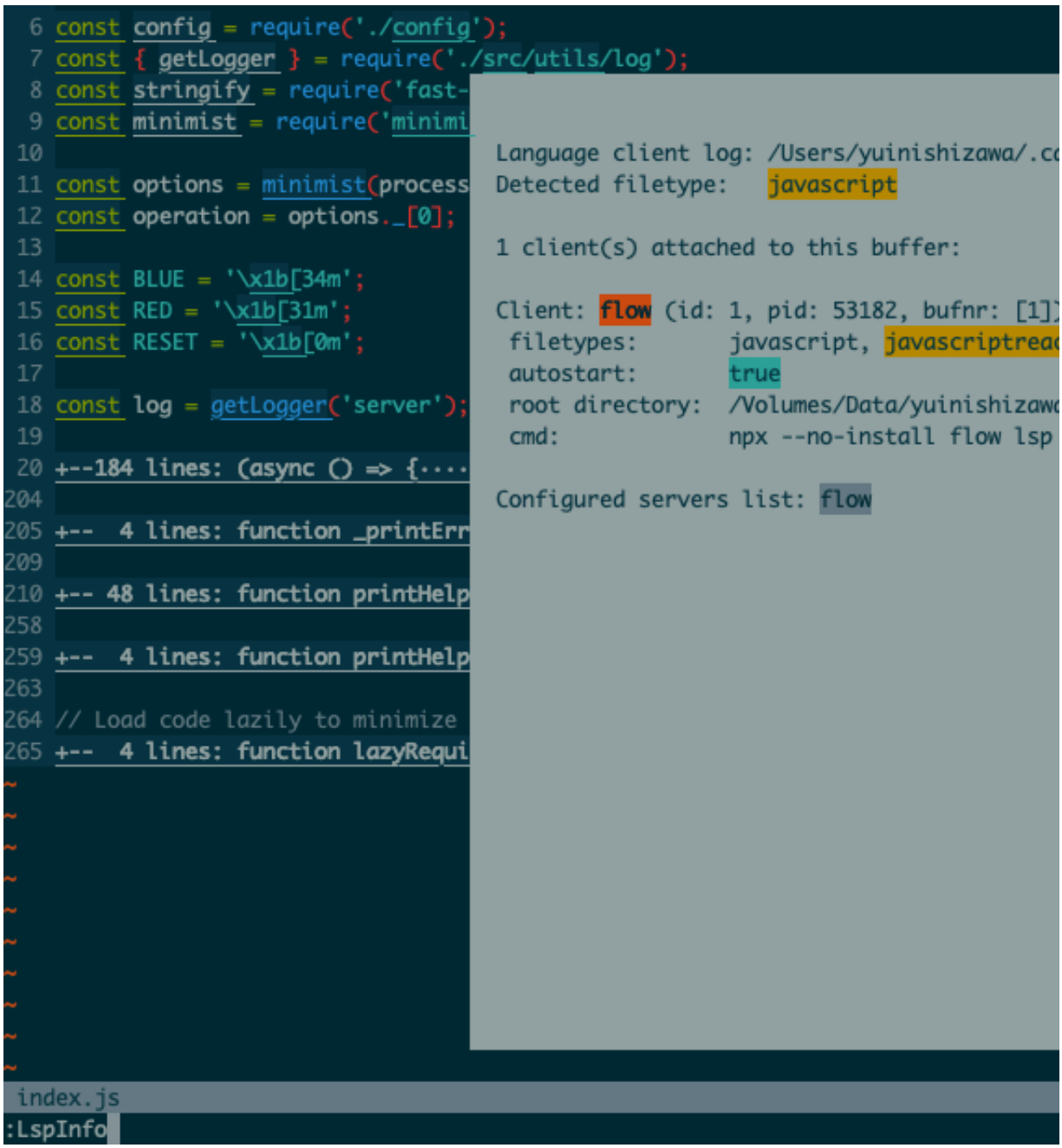
==========

Troubleshooting

Check if LSP is running

Use :LspInfo command to check if client is running

```
:LspInfo
```



[ERROR] cmp_nvim_lsp fails due to LSP error "nvim/runtime/lua/vim/lsp/client.lua:650: bufnr: expected number, got function"

Error like

```
Error detected while processing TextChangedI Autocommands for "*":
Error executing lua callback: ...eovim/0.10.4_1/share/nvim/runtime/
lua/vim/lsp/client.lua:650: bufnr: expected number, got function
stack traceback:
        [C]: in function 'error'
        vim/shared.lua: in function 'validate'
        ...eovim/0.10.4_1/share/nvim/runtime/lua/vim/lsp/
client.lua:650: in function 'resolve_bufnr'
        ...eovim/0.10.4_1/share/nvim/runtime/lua/vim/lsp/
client.lua:685: in function 'request'
        ...wa/.vim/plugged/cmp-nvim-lsp/lua/cmp_nvim_lsp/
source.lua:144: in function '_request'
        ...wa/.vim/plugged/cmp-nvim-lsp/lua/cmp_nvim_lsp/
source.lua:72: in function 'complete'
        ...rs/yuinishizawa/.vim/plugged/nvim-cmp/lua/cmp/
```

```
source.lua:342: in function 'complete'
        /Users/yuinishizawa/.vim/plugged/nvim-cmp/lua/cmp/
core.lua:308: in function 'complete'
        /Users/yuinishizawa/.vim/plugged/nvim-cmp/lua/cmp/
core.lua:178: in function 'callback'
        /Users/yuinishizawa/.vim/plugged/nvim-cmp/lua/cmp/
core.lua:238: in function 'autoindent'
        /Users/yuinishizawa/.vim/plugged/nvim-cmp/lua/cmp/
core.lua:170: in function 'on_change'
        /Users/yuinishizawa/.vim/plugged/nvim-cmp/lua/cmp/
init.lua:372: in function 'callback'
        ...ishizawa/.vim/plugged/nvim-cmp/lua/cmp/utils/
autocmd.lua:53: in function 'emit'
        ...ishizawa/.vim/plugged/nvim-cmp/lua/cmp/utils/
autocmd.lua:14: in function <...ishizawa/.vim/plugged/nvim-cmp/lua/
cmp/utils/autocmd.lua:13>
```

Problem
Client seems to be passing itself as the parameter

client.request(method, params, ...)

becomes

client.request(client, method, params, ...) due to method wrapper

Workaround

In cmp-nvim-lsp/lua/cmp_nvim_lsp/source.lua

```
source._request = function(self, method, params, callback)
  if self.request_ids[method] ~= nil then
    self.client:cancel_request(self.request_ids[method])
    self.request_ids[method] = nil
  end
  local _, request_id

  -- WORKAROUND
  --
  -- In nvim/runtime/lua/vim/lsp/client.lua
  --
  --    self._request = Client._request
  --    self.request = method_wrapper(self, Client._request)
  --
  --  and use _request instead
  --
  _, request_id = self.client:_request(method, params,
function(arg1, arg2, arg3)
  -- _, request_id = self.client:request(method, params,
function(arg1, arg2, arg3)
    if self.request_ids[method] ~= request_id then
      return
```

```
  end
  self.request_ids[method] = nil

...
end
```

In neovim/<version>/share/nvim/runtime/lua/vim/lsp/client.lua

```
-- WORKAROUND
self._request = Client._request
self.request = method_wrapper(self, Client._request)
self.request_sync = method_wrapper(self, Client._request_sync)
```